

System and Network Security

Dr. Ashok Kumar Das

Professor

Center for Security, Theory and Algorithmic Research
International Institute of Information Technology, Hyderabad

E-mail: *ashok.das@iiit.ac.in*

URL: <https://www.iiit.ac.in/faculty/ashok-kumar-das/>
<https://sites.google.com/view/iitkgpkdas/>

Application Layer Security: Electronic Mail (E-mail) Security

- As a default, PGP compresses the message after applying the signature but before encryption. This has the benefit of saving space both for e-mail transmission and for file storage.
- The placement of the compression algorithm, indicated by Z for compression and Z^{-1} for decompression is critical.
- The signature is generated before compression for two reasons:
 - ▶ (a) It is preferable to sign an uncompressed message so that one can store only the uncompressed message together with the signature for future verification.
Thus, if one signed a compressed document, it would be necessary either to store a compressed version of the message for later verification or to recompress the message when verification is required.
 - ▶ (b) Applying the hash function and signature after compression would constrain all PGP implementation to the same version of the compressed algorithm.

- Message encryption is applied after compression to strengthen cryptographic security. Because the compressed message has less redundancy than the original plaintext, cryptanalysis is more difficult.
- The compression algorithm used is ZIP.

- When PGP is used, at least part of the block to be transmitted is encrypted.
- If only the signature service is used, then the message digest is encrypted (with the sender's private key).
- If the confidentiality service is used, the message plus signature (if present) are encrypted (with a one-time symmetric key).
- Thus, part or all of the resulting block consists of a stream of arbitrary 8-bit octets.
- However, many electronic mail systems only permit the use of blocks consisting of ASCII text. To accommodate this restriction, PGP provides the service of converting the raw 8-bit binary stream to a stream of printable ASCII characters.
- The scheme used for this purpose is radix-64 conversion. Each group of three octets of binary data is mapped into four ASCII characters. This format also appends a CRC to detect transmission errors.

This technique maps arbitrary binary input into possible character output.

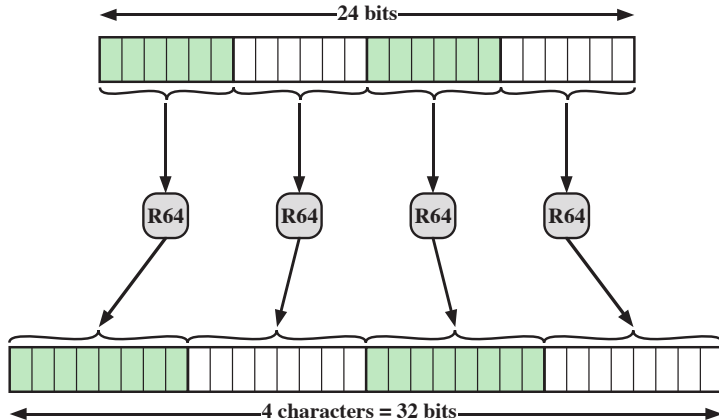
The form of encoding has the following relevant characteristics:

- The range of the function is a character set that is universally representable at all sites, not a specific binary encoding of that character set. Thus, the characters themselves can be encoded into whatever form is needed by a specific system.
- The character set consists of 64 printable characters, one of which is used for padding. With $2^6 = 64$ available characters, each character can be used to represent 6 bits of input.
- No control characters (i.e., non-printing characters) are included in the set.
- The hyphen character (“-”) is not used. This character has significance in the RFC 822 format and should therefore be avoided.

Radix-64 Encoding Table

6-Bit Value	Character Encoding	6-Bit Value	Character Encoding	6-Bit Value	Character Encoding	6-Bit Value	Character Encoding
0	A	16	Q	32	g	48	w
1	B	17	R	33	h	49	x
2	C	18	S	34	i	50	y
3	D	19	T	35	j	51	z
4	E	20	U	36	k	52	0
5	F	21	V	37	l	53	1
6	G	22	W	38	m	54	2
7	H	23	X	39	n	55	3
8	I	24	Y	40	o	56	4
9	J	25	Z	41	p	57	5
10	K	26	a	42	q	58	6
11	L	27	b	43	r	59	7
12	M	28	c	44	s	60	8
13	N	29	d	45	t	61	9
14	O	30	e	46	u	62	+
15	P	31	f	47	v	63	/
						(pad)	=

Printable Encoding of Binary Data into Radix-64 Format



ASCII Table

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	MUL (null)	32	20	040	&#32; Space		64	40	100	&#64; @		96	60	140	&#96; `	
1	1	001	SOH (start of heading)	33	21	041	&#33; !		65	41	101	&#65; A		97	61	141	&#97; a	
2	2	002	STX (start of text)	34	22	042	&#34; "		66	42	102	&#66; B		98	62	142	&#98; b	
3	3	003	ETX (end of text)	35	23	043	&#35; #		67	43	103	&#67; C		99	63	143	&#99; c	
4	4	004	EOT (end of transmission)	36	24	044	&#36; \$		68	44	104	&#68; D		100	64	144	&#100; d	
5	5	005	ENQ (enquiry)	37	25	045	&#37; %		69	45	105	&#69; E		101	65	145	&#101; e	
6	6	006	ACK (acknowledge)	38	26	046	&#38; &		70	46	106	&#70; F		102	66	146	&#102; f	
7	7	007	BEL (bell)	39	27	047	&#39; '		71	47	107	&#71; G		103	67	147	&#103; g	
8	8	010	BS (backspace)	40	28	050	&#40; (		72	48	110	&#72; H		104	68	150	&#104; h	
9	9	011	TAB (horizontal tab)	41	29	051	&#41;)		73	49	111	&#73; I		105	69	151	&#105; i	
10	A	012	LF (NL line feed, new line)	42	2A	052	&#42; *		74	4A	112	&#74; J		106	6A	152	&#106; j	
11	B	013	VT (vertical tab)	43	2B	053	&#43; +		75	4B	113	&#75; K		107	6B	153	&#107; k	
12	C	014	FF (NP form feed, new page)	44	2C	054	&#44; ,		76	4C	114	&#76; L		108	6C	154	&#108; l	
13	D	015	CR (carriage return)	45	2D	055	&#45; -		77	4D	115	&#77; M		109	6D	155	&#109; m	
14	E	016	SO (shift out)	46	2E	056	&#46; .		78	4E	116	&#78; N		110	6E	156	&#110; n	
15	F	017	SI (shift in)	47	2F	057	&#47; /		79	4F	117	&#79; O		111	6F	157	&#111; o	
16	10	020	DLE (data link escape)	48	30	060	&#48; 0		80	50	120	&#80; P		112	70	160	&#112; p	
17	11	021	DC1 (device control 1)	49	31	061	&#49; 1		81	51	121	&#81; Q		113	71	161	&#113; q	
18	12	022	DC2 (device control 2)	50	32	062	&#50; 2		82	52	122	&#82; R		114	72	162	&#114; r	
19	13	023	DC3 (device control 3)	51	33	063	&#51; 3		83	53	123	&#83; S		115	73	163	&#115; s	
20	14	024	DC4 (device control 4)	52	34	064	&#52; 4		84	54	124	&#84; T		116	74	164	&#116; t	
21	15	025	NAK (negative acknowledge)	53	35	065	&#53; 5		85	55	125	&#85; U		117	75	165	&#117; u	
22	16	026	SYN (synchronous idle)	54	36	066	&#54; 6		86	56	126	&#86; V		118	76	166	&#118; v	
23	17	027	ETB (end of trans. block)	55	37	067	&#55; 7		87	57	127	&#87; W		119	77	167	&#119; w	
24	18	030	CAN (cancel)	56	38	070	&#56; 8		88	58	130	&#88; X		120	78	170	&#120; x	
25	19	031	EM (end of medium)	57	39	071	&#57; 9		89	59	131	&#89; Y		121	79	171	&#121; y	
26	1A	032	SUB (substitute)	58	3A	072	&#58; :		90	5A	132	&#90; Z		122	7A	172	&#122; z	
27	1B	033	ESC (escape)	59	3B	073	&#59; ;		91	5B	133	&#91; [		123	7B	173	&#123; {	
28	1C	034	FS (file separator)	60	3C	074	&#60; <		92	5C	134	&#92; \		124	7C	174	&#124; 	
29	1D	035	GS (group separator)	61	3D	075	&#61; =		93	5D	135	&#93;]		125	7D	175	&#125; }	
30	1E	036	RS (record separator)	62	3E	076	&#62; >		94	5E	136	&#94; ^		126	7E	176	&#126; ~	
31	1F	037	US (unit separator)	63	3F	077	&#63; ?		95	5F	137	&#95; _		127	7F	177	&#127; DEL	

Source: www.LookupTables.com

Problem (Encoding): Consider the 24-bit raw text sequence 00100011 01011100 10010001, which can be expressed in hexadecimal as 235C91.

- We arrange this input in blocks of 6 bits:
001000 110101 110010 010001
- The extracted 6-bit decimal values are 8, 53, 50, 17. Looking these up in encoding table yields the radix-64 encoding as the following characters: l1yR.
- If these characters are stored in 8-bit ASCII format with parity bit set to zero, we have
01001001 00110001 01111001 01010010
In hexadecimal, this is 49317952.

Radix-64 Encoding Example

The following table provides a summary.

Input Data	
Binary representation	00100011 01011100 10010001
Hexadecimal representation	235C91
Radix-64 Encoding of Input Data	
Character representation	I1yR
ASCII code (8 bit, zero parity)	01001001 00110001 01111001 01010010
Hexadecimal representation	49317952

Problem (Decoding): Consider the 32-bit radix-64 encoding of an input data in hexadecimal be 49317952.

- The ASCII code (8 bits, zero parity) of the encoding input becomes 01001001 00110001 01111001 01010010
- The values of 01001001, 00110001, 01111001, and 01010010 are 73, 49, 121, and 82, respectively, which have further their corresponding ASCII values as

llyR

- Looking up the encoding table, we have the corresponding 6-bit values of l, l, y, and R as 8, 53, 50, and 17, respectively.
- Finally, we have the 24-bit raw input data as:
001000 110101 110010 010001
i.e., 00100011 01011100 10010001

- E-mail facilities often are restricted to a maximum message length. For example, many of the facilities accessible through the Internet impose a maximum length of 50,000 octets.
- Any message longer than that must be broken up into smaller segments, each of which is mailed separately.
- To accommodate this restriction, PGP automatically subdivides a message that is too large into segments that are small enough to send via e-mail.
- The segmentation is done after all of the other processing, including the radix-64 conversion. Thus, the session key component and signature component appear only once, at the beginning of the first segment.
- At the receiving end, PGP must strip off all e-mail headers and reassemble the entire original block before performing the steps.

Thank You !!!